IT UNIVERSITY OF COPENHAGEN

Game Programming 25

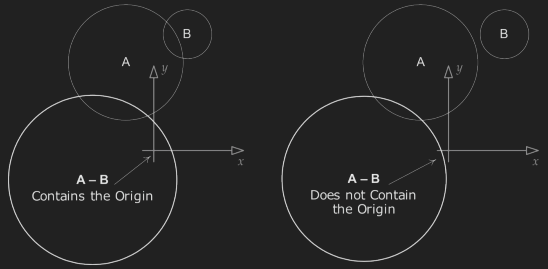
# L03: Coordinate Systems

Chris Carvelli

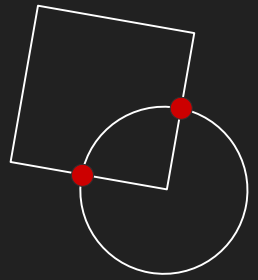
# Changelog

1.1: corrected structure of exercise libraries explanation slide

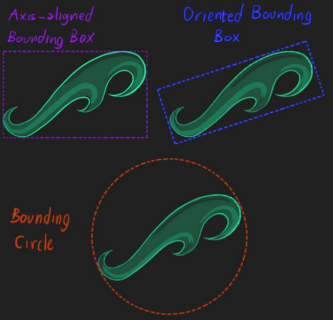
# Recap



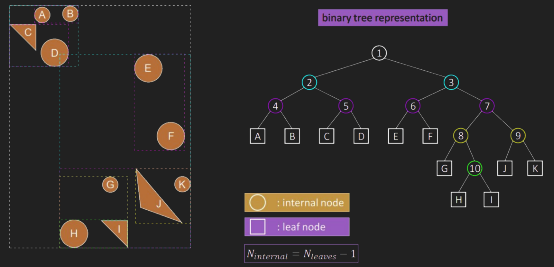

GJK algorithm



terminology




collider types




optimizations

# Agenda

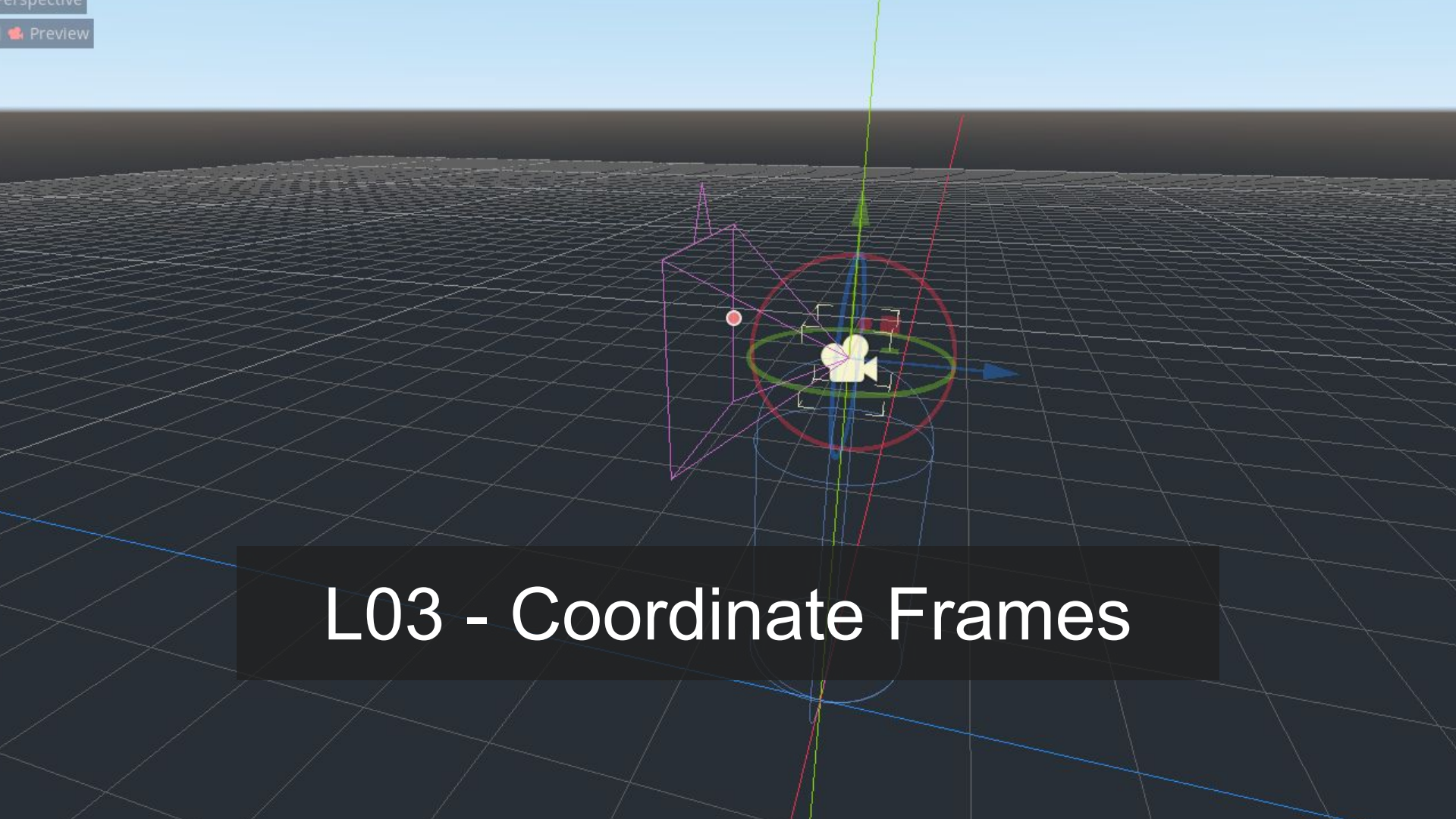
1. Coordinate Frames
2. 2D Transforms
3. C++ Building Pipeline

# Early feedback results

(delayed?)

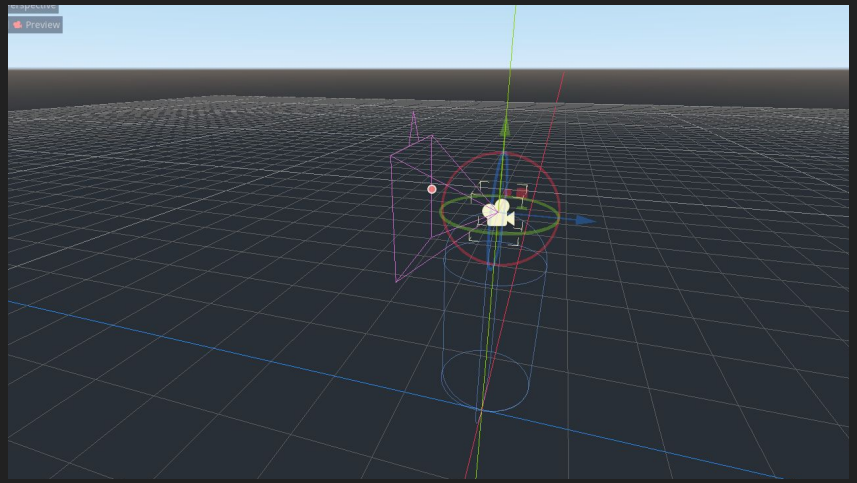

Game Programming Stories
wanted!

# Exercise 02 review



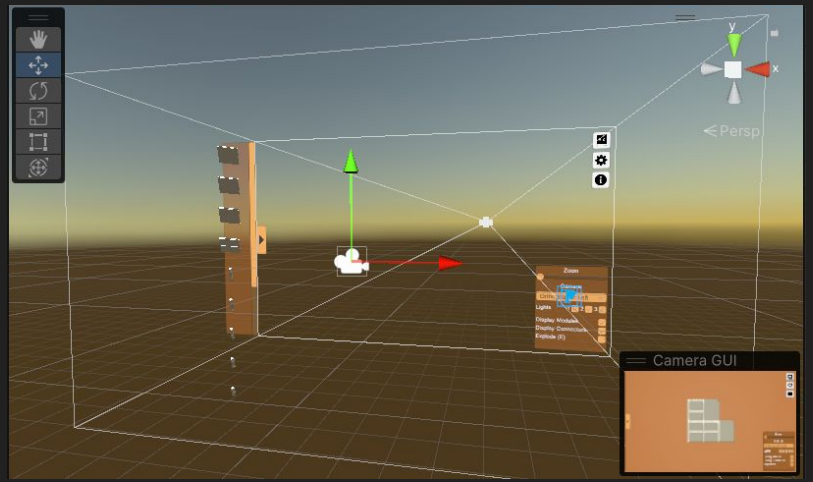

# L03 - Coordinate Frames

# Problem statement: How do we…




- …decouple our game from the screen size?
- …make our game resolution-independent?

- …simplify the math and the code involved in placing objects in the world?
- …make life easier for everyone?



# Agenda

1. Coordinate Frames
2. 2D Transforms
3. C++ Building Pipeline

# What are we trying to tackle?

```
static void sprite_render(
   SDLContext* context, vec2f position, vec2f size, Sprite* sprite
)
{
    SDL_FRect dst_rect;
    dst_rect.w = size.x;
    dst_rect.h = size.y;
    dst_rect.x = position.x - dst_rect.w * sprite->pivot.x;
    dst_rect.y = position.y - dst_rect.h * sprite->pivot.y;

    // ...
}
```

1. texture center differs from sprite center

```
// clamp player position
entity_player->position.x = SDL_clamp(
    entity_player->position.x,
    entity_player->rect.w / 2,
    context->window_w - entity_player->rect.w / 2
);
```

2. one screen is too small for an entire game world

```
const float entity_size_world = 64;
const float entity_size_texture = 128;
const float player_speed = entity_size_world * 5;
const float player_collision_radius = 16;
const int   player_sprite_coords_x = 4;
const int   player_sprite_coords_y = 0;
const float asteroid_speed_min = entity_size_world * 2;
const float asteroid_speed_range = entity_size_world * 4;
```

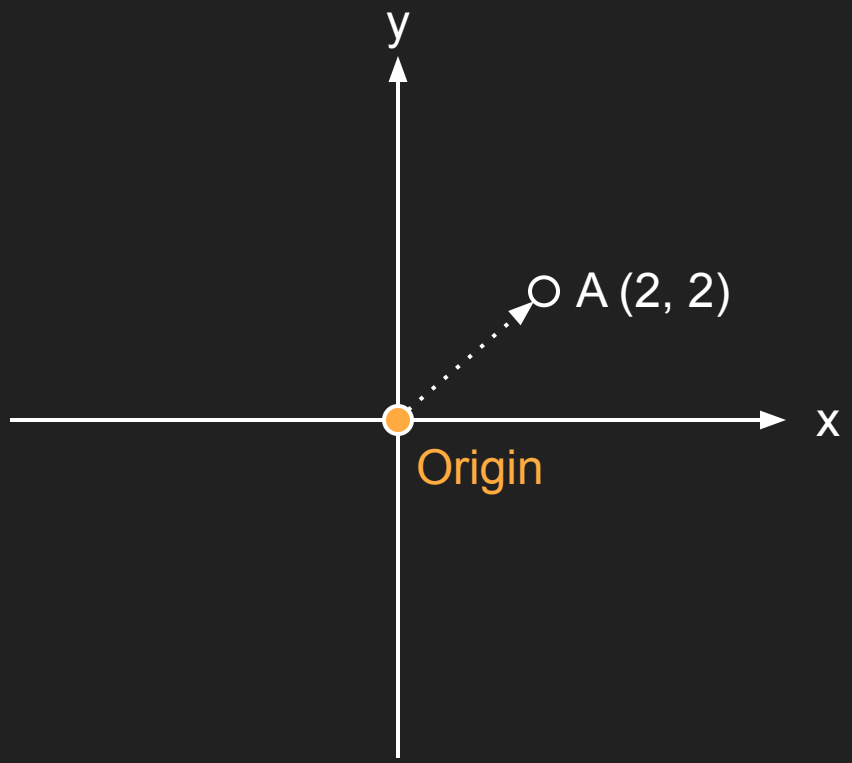
3. messy to have game code depend directly on texture size, screen size and so on

# What do we want?

1. game code independent from resources
    a. textures are rendered at the correct size/aspect ratio, without constant manual intervention
    b. changing in window/monitor size do not affect the game
2. game code free to reason about space in whatever way is more convenient for design
3. a mechanism to render stuff outside of the screen

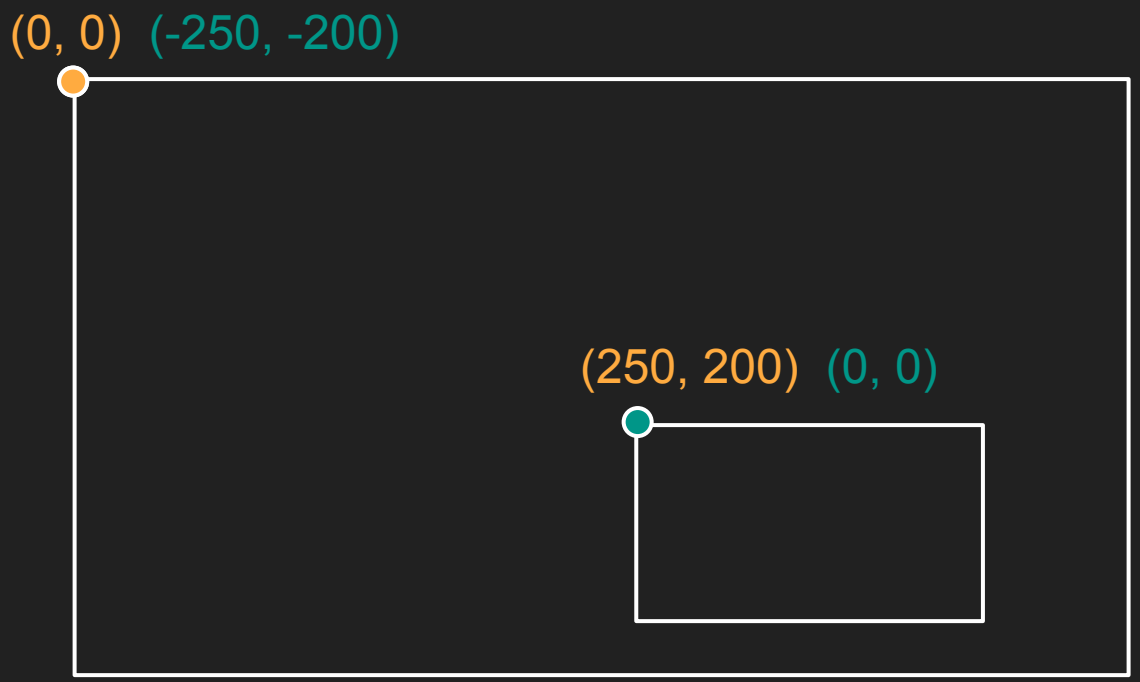
# Frames of reference

1. Positions in space are always relative to something else
    a. Example: where am I? (Relative to room, relative to CPH, relative to the sun)
2. Different tasks are easier in different frames of reference
    a. Example: rotate around origin VS rotate around point



# Moving between frames of reference

It’s always possible to move between frames of reference, it’s the inverse operation!



# Transformation functions

We will focus mostly on transformations that are easily invertible

1. Translate
2. Rotate
3. Scale

# Coordinate Systems we will use

1. object
2. world
3. camera
4. screen

# Coordinate Systems we will use

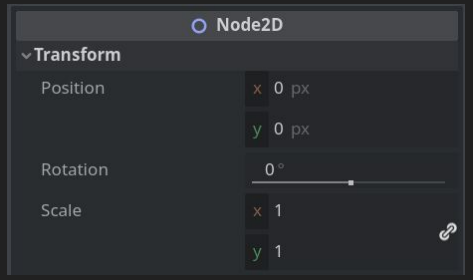
1. object (texture)
   a. up: negative Y
   b. right: positive X
2. world (our game logic)
   a. up: positive Y
   b. right: positive X
3. camera
   a. up: positive Y
   b. right: positive X
4. screen
   a. up: negative Y
   b. right: positive X

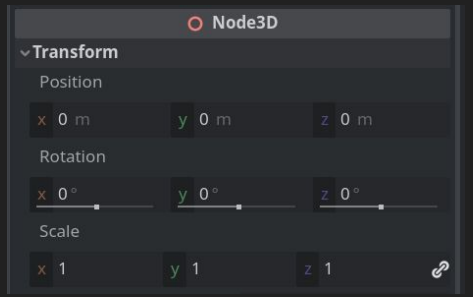
# In Practice

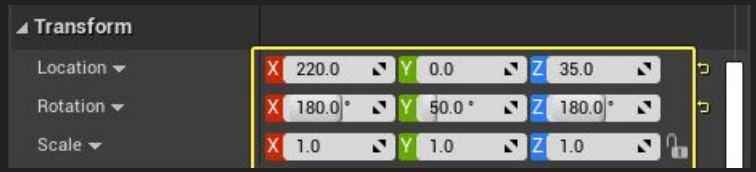
# Agenda

1. Coordinate Frames
2. 2D Transforms
3. C++ Building Pipeline

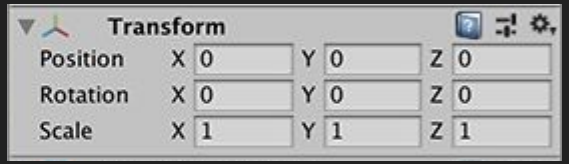
# What is a transform?







# Transform representations

1. Position, rotation scale (today's choice)
    a. Intuitive
    b. A bit more cumbersome
    c. A bit less efficient
2. Transform matrices (for 3D exercises)
    a. They have their specialized math
    b. Easier to use after understanding them
    c. More efficient

```
struct Transform
{
    vec2f position;
    vec2f scale;
    float rotation;
};
```

# A note or rotations

More complex that it seems

1. How to do it efficiently
2. How to do it conveniently
3. How to do it unambiguously (3D)

For 2D, we have two main options

1. angle (degree or radians)
   simple but slow (need to do a bung of trigonometry every time we use it)
2. sin/cos
   performant but complex (trigonometry done only once, but the we need to use it correctly when we do computations)

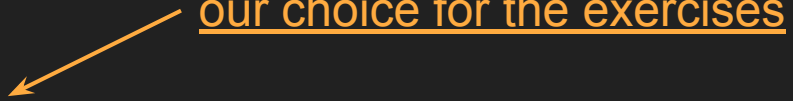
# A note or rotations

More complex that it seems

1. How to do it efficiently
2. How to do it conveniently
3. How to do it unambiguously (3D)

For 2D, we have two main options

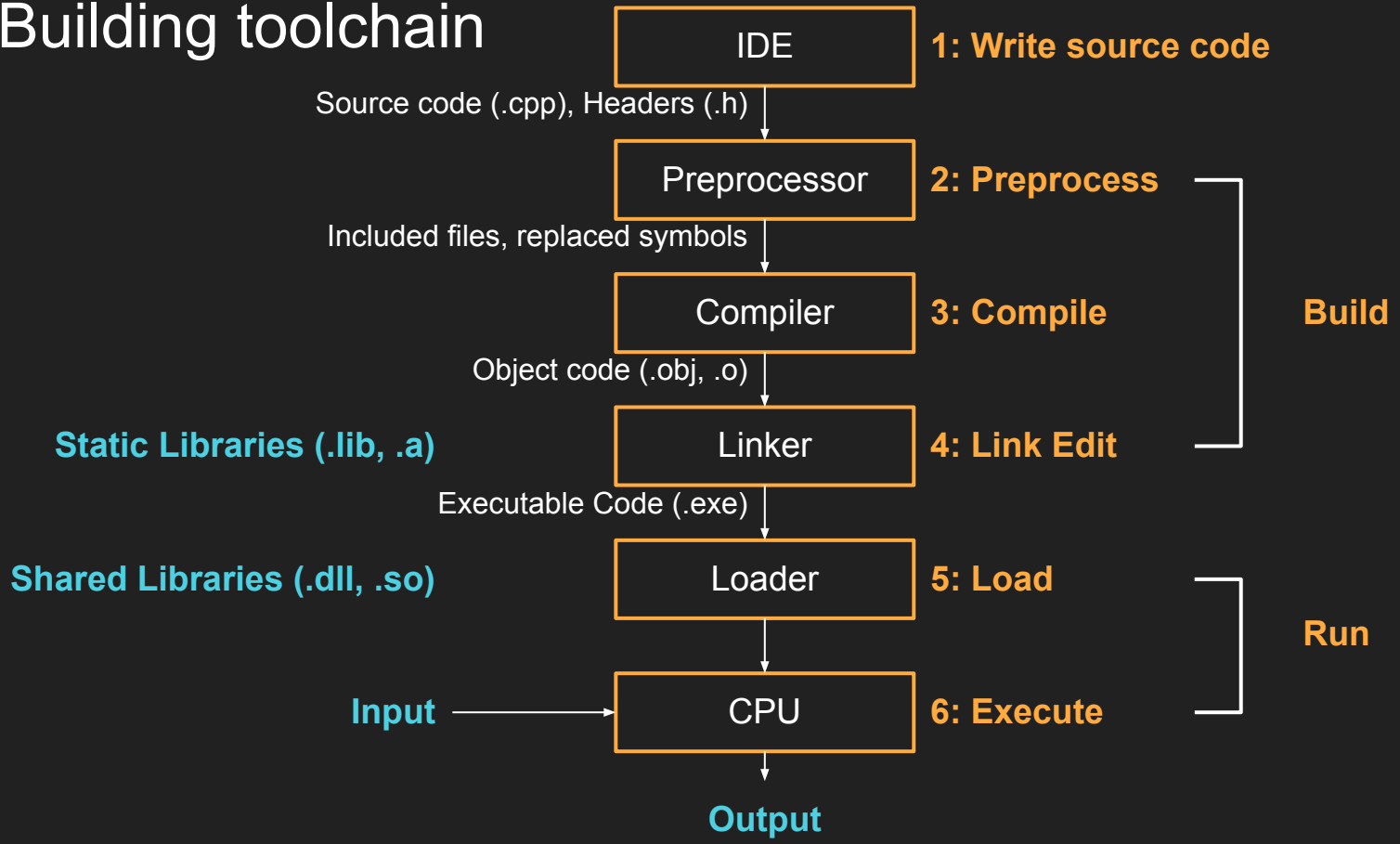
1. angle (degree or radians)
   simple but slow (need to do a bung of trigonometry every time we use it)
2. sin/cos
   performant but complex (trigonometry done only once, but the we need to use it correctly when we do computations)



# Agenda

1. Coordinate Frames
2. 2D Transforms
3. C++ Building Pipeline

# C++ Building toolchain

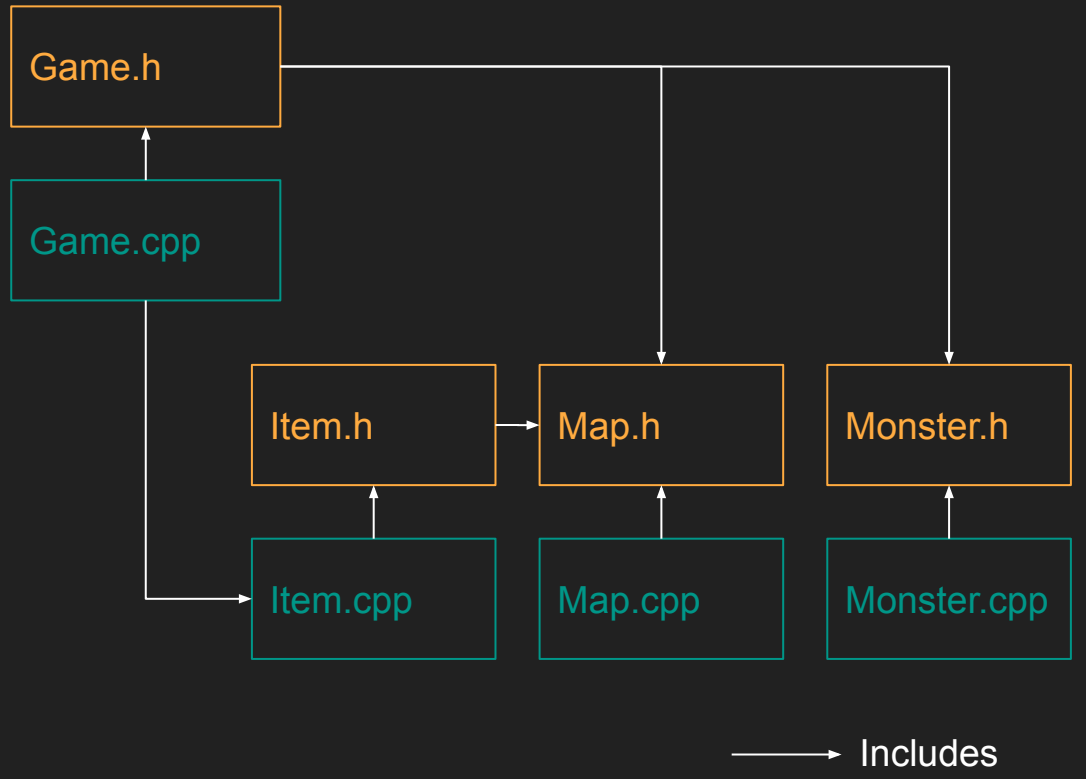


# Classic C++ build

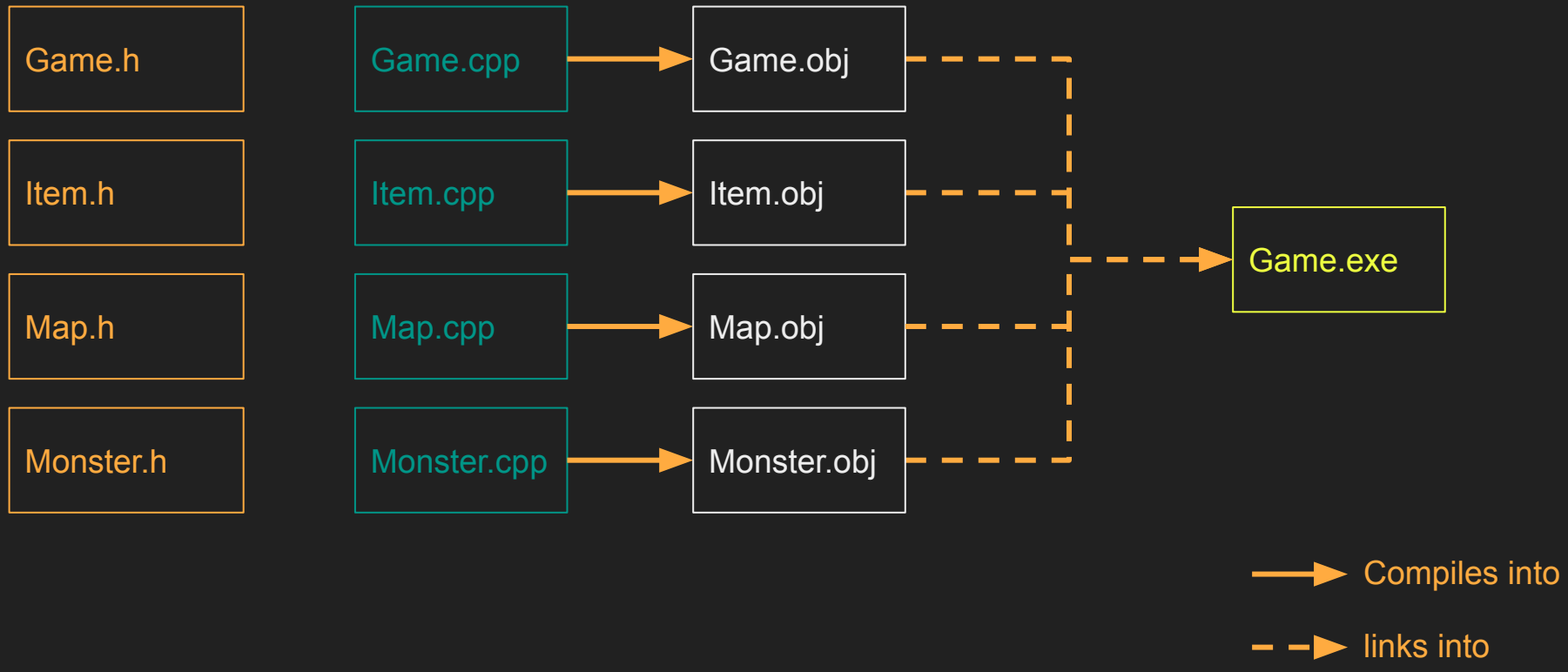
The header files (.h, .hpp, hxx) contain declarations that need to be shared among multiple source files.

- data types (struct and class types)
- function declarations
- constant values
- enumeration types

The source files (.c, cpp, .cxx) contain the actual executable functions and methods of the program, as well as any global data.



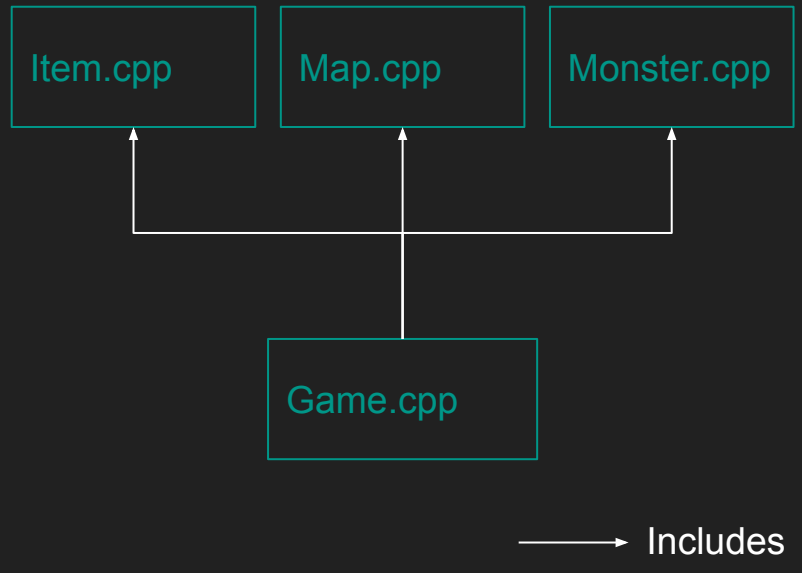
# Classic C++ build - separate compilation



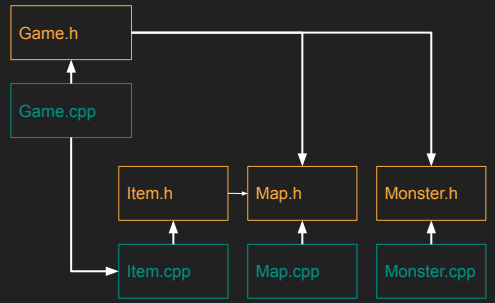
# Unity build

Due to a number of factors (mostly inefficient preprocessing and issues with incremental linking), people developed a new approach called Unity Builds (as in “unified”).

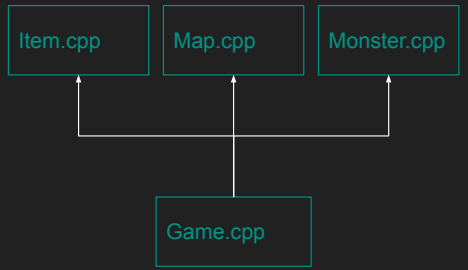
The idea is to have one single compilation unit, and include all code in there (which extension you want to use doesn’t really matter)



# Classic VS Unity builds




- Pros
  - well known and understood (historic choice)
  - maps well to OOP
- Cons
  - can be very slow to compile (especially C++, especially big codebases)
  - requires dedicated build systems (on top of the compiler!) to keep track of everything
  - difficult for beginners to wrap their head around it




- Pros
  - very fast to compile
  - no need to complex build systems
- Cons
  - code tends to spiral a bit if not tende
  - difficult for beginners to wrap their head around it

# Exercise’s approach

```
#ifndef ITU_LIB_RENDER_HPP
#define ITU_LIB_RENDER_HPP

#include <SDL3/SDL_render.h>
#include <itu_common.hpp>

// "public" defines
// "public" data types
// "public" functions' declarations

#if defined ITU_LIB_RENDER_IMPLEMENTATION || defined ITU_UNITY_BUILD

// "private" defines
// "private" data types
// "private" functions
// "public" functions' implementations

# endif //ITU_LIB_RENDER_IMPLEMENTATION

#endif // ITU_LIB_RENDER_HPP
```

# Today’s exercise